



ARISTOTLE
UNIVERSITY
OF THESSALONIKI

Αναφορά: K-Nearest Neighbors με OpenMP και OpenBLAS

Ατομική εργασία

Μάθημα: Ενσωματωμένα Συστήματα Πραγματικού Χρόνου

1. Εισαγωγή

Στην Εργασία 0 κληθήκαμε να επιλύσουμε το πρόβλημα εύρεσης των K- κοντινότερων γειτόνων (KNN) για ένα μεγάλο σετ δεδομένων $N = 20.000$ σημεία των 10 διαστάσεων.

Αν προσπαθούσαμε να λύσουμε αυτό το πρόβλημα κατευθείαν, υπολογίζοντας σειριακά την απόσταση κάθε σημείου προς όλα τα υπόλοιπα, θα δημιουργούσαμε έναν τεράστιο πίνακα. Κάτι τέτοιο θα γέμιζε αμέσως τη μνήμη RAM και θα έκανε την εκτέλεση εξαιρετικά αργή. Στόχος μας λοιπόν ήταν να

γράψουμε έναν έξυπνο κώδικα σε C που θα υπολογίζει τις αποστάσεις γρήγορα, θα βρίσκει τους γείτονες χωρίς περιττές πράξεις και στην ουσία θα διαιρεί τον πίνακα σε πολλά μικρότερα blocks ώστε να τρέχουν παράλληλα.

2. Ανάλυση της Υλοποίησης στον Κώδικα

Ο κώδικας που γράψαμε (στο αρχείο Ergasia0.c) στηρίζεται σε τρεις βασικές ιδέες:

A) Γρήγορα Μαθηματικά (OpenBLAS)

Αντί να χρησιμοποιήσουμε κλασικά for loops για να βρούμε την Ευκλείδεια απόσταση, χρησιμοποιήσαμε τον τύπο πινάκων $D = C^2 - 2CQ^T + Q^2$ που δόθηκε. Ο πολλαπλασιασμός των πινάκων αποτελεί το πιο χρονοβόρο κομμάτι. Για να τον επιταχύνουμε, έγινε χρήση της συνάρτησης `cblas_dgemm` της OpenBLAS.

B) Quick-Select

Αφού βρούμε τις αποστάσεις, πρέπει να βρούμε τους 5 μικρότερους γείτονες. Ο προφανής τρόπος θα ήταν να ταξινομήσουμε τα πάντα από το μικρότερο στο μεγαλύτερο. Όμως, αντί να κάνουμε μια πλήρη ταξινόμηση που καθυστερεί το πρόγραμμα, φτιάξαμε τον δικό μας αλγόριθμο Quick-Select. Αυτός ο αλγόριθμος ταξινομεί μερικώς τα στοιχεία και σταματάει αμέσως μόλις βρει τους 5 μικρότερους. Έτσι δεν χάνουμε χρόνο ταξινομώντας αποστάσεις που δεν μας νοιάζουν.

Γ) Παράλληλισμός OpenMP και Διαχείριση Μνήμης

Για να μην σκάσει το πρόγραμμα από έλλειψη μνήμης, ορίσαμε να δουλεύει σε μικρά μπλοκ των 1000 σημείων.

Φτιάξαμε την αναδρομική συνάρτηση `approx_knn`, η οποία κοιτάει τα δεδομένα και τα κόβει στη μέση. Εδώ χρησιμοποιήσαμε το OpenMP: βάζοντας την εντολή `#pragma omp task`, αναθέσαμε το κάθε μισό σε διαφορετικό thread του επεξεργαστή. Ο υπολογιστής τρέχει όλα τα κομμάτια ταυτόχρονα και καλεί τα βαριά μαθηματικά της OpenBLAS μόνο όταν τα δεδομένα έχουν γίνει αρκετά μικρά ώστε να χωράνε άνετα στη μνήμη.

3. Αποτελέσματα και Συμπεράσματα

Τρέχοντας το πρόγραμμα στο τερματικό για 20.000 σημεία και ζητώντας τους 5 πιο κοντινούς γείτονες, πήραμε το εξής αποτέλεσμα:

Ο συνολικός χρόνος εκτέλεσης ήταν μόλις 0.397 δευτερόλεπτα.

Από αυτό το νούμερο βγαίνει ένα πολύ ξεκάθαρο, πρακτικό συμπέρασμα: Η ωμή επεξεργαστική ισχύς δεν αρκεί από μόνη της αν ο κώδικας δεν είναι δομημένος σωστά. Ο χρόνος αυτός επιτεύχθηκε επειδή:

1. Λύσαμε το πρόβλημα της μνήμης διαιρώντας τα δεδομένα σε μικρότερα κομμάτια.
2. Μοιράσαμε τον φόρτο εργασίας σε όλους τους πυρήνες παράλληλα μέσω του OpenMP.
3. Βρήκαμε τους γείτονες χωρίς περιττές ταξινομήσεις.

Η εργασία μάς έδειξε στην πράξη πως όταν συνδυάζουμε τον παράλληλο προγραμματισμό με έτοιμες, βελτιστοποιημένες βιβλιοθήκες, μπορούμε να τρέξουμε βαριούς μαθηματικούς αλγόριθμους σε απίστευτα γρήγορους χρόνους, κάτι που είναι απολύτως αναγκαίο σε τέτοια συστήματα.